

Revisión a fondo — Trade

Auditoría de código + visual/UX combinada · 2026-08-04 · APPS/trade

Resumen ejecutivo: Trade está funcionalmente sólido y bien probado — 69/69 tests automatizados corridos en vivo para esta auditoría, dependencias 100% al día, manejo de errores robusto en los 6 módulos, y el gasto ya optimizado con prompt caching verificado con llamadas reales a la API. Pero todavía no está lista para entregarse: falta el empaquetado a .exe (bloqueante real — hoy necesita Python instalado en la PC de Sebas), el toque personal de "regalo" (nombre + mensaje de bienvenida siguen siendo placeholders), y hay un hallazgo de accesibilidad real y medido con precisión (contraste de las burbujas de chat muy por debajo del mínimo). Nada de esto requiere rehacer código — son ajustes puntuales sobre una base sólida.

Auditoría de código

 ~85%

Auditoría visual/UX

 ~65%

1. Auditoría de código



BUGS

El chat puede quedar "trabado" si el modelo encadena muchas herramientas seguidas en un mismo turno Media

Qué es: `main.py` corta el intercambio con la API después de 5 llamadas de herramienta seguidas dentro de un mismo turno (memoria, búsqueda web, registrar operación, agregar nota, calcular riesgo — hoy hay 5 herramientas activas). Si el corte llega justo cuando la última respuesta todavía tiene una herramienta pendiente sin resolver, esa conversación incompleta queda guardada en el historial.

Por qué importa: la API de Anthropic exige que toda herramienta pendiente tenga su resultado inmediatamente después en la conversación. Con el historial en ese estado, el *próximo* mensaje que mande Sebas muy probablemente falle con un error, y el chat quede sin responder hasta reiniciar la app — sin ningún aviso claro de qué pasó.

Acción sugerida: cuando se llegue al tope de intentos con una herramienta todavía pendiente, generar un resultado de "no se pudo completar" para esa herramienta antes de cortar, en vez de dejar el turno a medias.



PARA ELIMINAR

Revisión de imports en los 6 módulos (comparando cada nombre importado contra su uso real en el archivo): sin imports muertos ni código sin usar detectado.



MEJORAS RECOMENDADAS

Falta el empaquetado a .exe Alta

Qué es: no existe todavía un archivo `.spec` de PyInstaller para Trade (Pollito sí tiene el suyo, `Pollito.spec`).

Por qué importa: sin esto, Trade no se le puede entregar a Sebas como un ejecutable que simplemente abre — necesitaría tener Python y todas las dependencias instaladas en su propia PC, lo que rompe la idea original de "regalo listo para usar".

Acción sugerida: armar el .spec siguiendo el mismo patrón que Pollito y generar el .exe una vez que el resto de los ajustes de esta auditoría estén resueltos.

Falta el toque personal de "regalo" Alta

Qué es: APP_NAME y el mensaje de bienvenida siguen siendo placeholders genéricos (hay un TODO explícito dejado en config.py).

Por qué importa: la app funciona perfecto, pero no "se siente" todavía como el regalo pensado para Sebas — Pollito sí tiene ese detalle (su mensaje especial).

Acción sugerida: definir con el usuario el nombre final y si hay una ocasión/motivo para un mensaje de bienvenida especial, igual que se hizo con Pollito.

Sin protección si se abre la app dos veces a la vez Baja

Qué es: no hay un lock entre procesos para operaciones.xlsx — si Trade.exe se abre dos veces al mismo tiempo, la última escritura gana en silencio.

Por qué importa: riesgo bajo para una app de un solo usuario en su propia PC — ya se había aceptado este riesgo al construir la bitácora, se repite acá por completitud del informe.

Acción sugerida: no es urgente; revisar solo si en algún momento se observa un problema real.

✓ LO QUE YA FUNCIONA BIEN

69/69 tests automatizados pasando (corridos en vivo para esta auditoría), con cobertura real de casos límite: racha de pérdidas, precios de prompt caching, protección contra path traversal en la memoria, condiciones de carrera en el gasto mensual.

Manejo de errores consistente en los 6 módulos: ninguna función deja pasar una excepción cruda hacia el loop de la API — siempre devuelve un mensaje legible para Sebas.

secreto.py correctamente excluido de git (verificado con `git check-ignore`) — la API key nunca se pidió ni se aceptó pegada en el chat.

Dependencias 100% al día con PyPI — `pip list --outdated` no encontró ninguna desactualizada (customtkinter, anthropic, pillow, openpyxl, pyinstaller).

Prompt caching implementado y verificado con llamadas reales a la API (`cache_read_input_tokens` confirmado funcionando) — reduce ~9x el costo del tramo repetido de cada llamada, estira bastante el presupuesto mensual.

Separación de responsabilidades limpia: cada módulo (memoria, bitácora, riesgo, uso mensual) hace una sola cosa y es testeable de forma aislada, siguiendo siempre el mismo patrón.

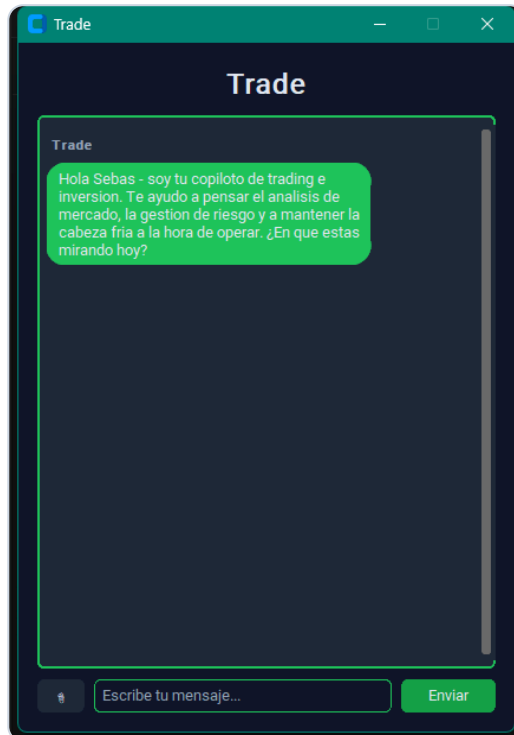
```
$ python -m pytest -q
..... [100%]
69 passed in 6.96s

$ pip list --outdated | grep -iE "customtkinter|anthropic|pillow|openpyxl|pyinstaller"
(sin resultados - todo al día)
```

```
$ git check-ignore -v secreto.py
APPS/trade/.gitignore:9:secreto.py      secreto.py
```

2. Auditoría visual / UX

App de escritorio (Python + customtkinter, no cubierta por las tablas de stack de ESTETIK que asumen Electron/Web/Expo) — se adaptaron los mismos criterios: se vio la app realmente renderizada y corriendo (no solo el código), con captura real incluida abajo.



Trade corriendo en vivo (PID capturado en la auditoría) — pantalla de bienvenida, sin interacción todavía

INCONSISTENCIA VISUAL

Sin ícono propio de la app Media

Qué es: se ve en la captura — la barra de título muestra un ícono genérico ("C" azul), no uno propio de Trade. Pollito sí tiene su propio ícono (assets/icons/icon.ico + lógica de aplicarlo en main.py).

Por qué importa: detalle chico pero afecta que se sienta como una app terminada y con marca propia, no un script.

Acción sugerida: crear un ícono simple (podría ir de la mano del "toque de regalo" pendiente en la auditoría de código) y aplicarlo con el mismo patrón que Pollito.

ACCESIBILIDAD

Las burbujas de chat no cumplen el contraste mínimo de accesibilidad Alta

Qué es: medido con precisión (fórmula oficial WCAG de contraste, no a ojo) sobre los colores reales de tema.py:

Combinación	Contraste	Mínimo AA	Resultado
Fondo principal / texto	14.48:1	4.5:1	Pasa
Fondo secundario / texto secundario	5.71:1	4.5:1	Pasa

Burbuja de Sebas (verde oscuro) / texto	2.67:1	4.5:1	Falla
Burbuja de Trade (verde claro) / texto	1.85:1	4.5:1	Falla

Por qué importa: son literalmente los dos colores de fondo de *cada mensaje* del chat — se nota en la captura que el texto sobre el verde claro cuesta más de leer, y en pantallas con brillo bajo o para alguien con baja visión puede ser realmente difícil.

Acción sugerida: oscurecer los dos verdes de las burbujas (o usar texto oscuro sobre ellas) hasta llegar a 4.5:1 — la combinación fondo secundario/texto secundario de la misma paleta ya da 5.71:1 y sirve de referencia de cuánto ajustar.

✦ MICROINTERACCIONES / UX

La barra de scroll queda visible sin hacer falta Baja

Qué es: se ve en la captura — con un solo mensaje corto y mucho espacio vacío debajo, la barra de scroll del área de chat igual aparece a la derecha.

Por qué importa: ruido visual chico pero evitable — Pollito resolvió exactamente este mismo problema (ocultar el scroll cuando no hace falta); Trade se simplificó sin esa lógica al construirse.

Acción sugerida: reusar el mismo mecanismo que ya existe en `skills/base.py` de Pollito para mostrar la barra solo cuando el contenido desborda.

✓ LO QUE YA FUNCIONA BIEN

El resto de la paleta cumple WCAG AA con margen amplio: fondo principal/texto da 14.48:1, fondo secundario/texto secundario da 5.71:1 (ambos medidos con precisión, no a ojo).

Layout limpio y consistente: tamaño de ventana fijo, padding uniforme, esquinas redondeadas consistentes entre burbujas, botones y el chip de adjuntar imagen.

El botón de adjuntar imagen (📎) y el campo de texto están bien diferenciados a simple vista, con placeholder claro y con buen contraste.

Campo de entrada y botón "Enviar" se deshabilitan visualmente mientras se espera la respuesta (revisado en código) — buen feedback de estado, evita el doble envío.

Si querés que se arreglen estos hallazgos: **PECAS** se encarga de los cambios de código/UI (empaquetado, ícono, ajuste de colores, ocultar el scroll) y **FIXA** del bug puntual del tope de reintentos — pero no se arranca a arreglar nada de esto sin que lo pidas explícitamente.